Date .....

Expt. No. ..... **OOPs** Page No. .....

* **OOPs** – It is a programming approach that organizes code around objects rather than func or logic.
An object is a real world entity that contains data & code.

→ OOP lang. support – Python, Java, C++, C, C#, Ruby, Dart.

* **Purpose of using oop concept –**
The aim of OOP is to implement real world entities like inheritance, Abstraction, Polymorphism in programming.
The main purpose of the OOP is to bind together the data and func$^n$ that operate on them so that no other part of the code can access this data except them.

* **Four main features of OOP.**
(I) Inheritance (II) Encapsulation (III) Polymorphism
(IV) Abstraction.

* **Encapsulation –**
- The wrapping up of data and function into a single unit (called class) is known as encapsulation.
- The data is not accessible to outside world, only those func$^n$ which are wrapped in the class can access it.

**Private Members**
These (attributes or methods) are accessible only within the class they are defined in.
It help the data protection and controlled access.

URBAN EDGE

Teacher's Signature : _____

## Public member

- These are accessible from outside the class, meaning any other code can read or use them.
- It provides a way for users to interact with the class's functionality from outside.

## * Abstraction

It is a process of hiding the unwanted & irrelevant data/details from the user.

It allows users to interact with an object without needing to understand its internal working and users can access that data which is necessary.

Key points -

- Hides complexity
- Simplifies interface
- Reduces error.

## * Polymorphism.

It means ability to take more than one form.

An operation may exhibit diff behaviour in diff instances.

Types - (I) compile time
(II) Runtime.

(I) compile time → Achieved through func$^n$/method overloading and operator overloading. The decision of which method to call at ~~run~~ compile time.

- Func$^n$/method - multiple method in the same class with same name but diff. parameters.
- Operator - It is the process of making an operator to exhibit diff behaviours in diff instance.

Date .....

Expt. No. ..... Page No. .....

(II) Runtime polymorphism – achieved through method overridding using Inheritance & virtual funcn.

- Method overridding – A derived class provides a specific implementation of a method that is already defined in its class.

## * Inheritance.

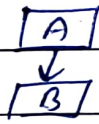
→ It is the process by which object of one class inherits/~~the~~ acquire the properties of object of another class.

→ It provides reusability means we can add additional features to an existing class without modifying it.

→ It supports the concept of hierarchial classification.

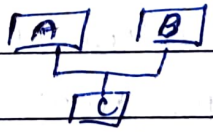
**Types**

(I) Single Inheritance – A derived class inherits from only one base class.

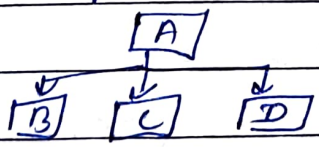



(II) Multi-level – A class is derived from a class which is already a derived class.

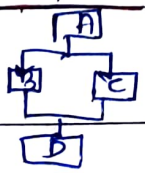

(III) Multiple – A derived class inherits from more than one class.




(IV) Hierarchal – Multiple classes inherits from single base class




(V) Hybrid Inheritance – A combination of two or more types of inheritance




URBAN EDGE

Teacher's Signature : _____

## New Operator.

The new operator allows dynamic memory allocation, creating object at runtime as needed, and is essential for using objects in prog. efficiently.

Key points- (I) Memory allocation
(II) constructor call
(III) Return a reference.

## Call by value

A copy of the actual argument value is passed to the funcn. The funcn works on this copy, so any changes made to this parameter do not affect the original argument.

- May be slower for larger data structure.

## Call by References -

The actual memory address of the argument is passed to the funcn
The funcn operates on the original value if any changes is made to the parameter, it reflects in the original value.

- It is more efficient.

## Inline funcn.

It is a funcn in which the compiler attempts to expand the funcn code at the point where funcn is called, instead of performing a normal funcn call.

→ When inline funcn is called compiler replaces the funcn call with the actual funcn
→ Improves efficiency for small funcn.
→ Performance boost.

Disadvantage - • Increased code size
• Limited use for large or complex funcn. (eg with loops or recursion)

Date .....

Expt. No. ..... Page No. .....

## Static Member

In C++, static members are members of a class that are shared by all objects of that class. This means that they exist only once, regardless of how many instances of class are created.

Types –

(I) Static Data Member –

- Declared with static keyword with the class.
- Shared by all objects of the class
- Initialized only once, usually outside the class definition
- Accessed using the class name or an object of the class.

(II) Static Member function –

- Declared with static keyword
- can be accessed without creating an object of the class
- Can only access static members of the class.

## static function

A static funcⁿ is a member funcⁿ of a class that is associated with the class itself rather than with an instance or object of the class.

This means that a static funcⁿ can be called without creating an instance of a class.

## delete operator

It is used to deallocate the memory that was previously allocated using the 'new' operator. It is crucial to use delete to release memory that's no longer needed.

URBAN EDGE

Teacher's Signature : _____

## * Recursion.

- when func$^n$ is called within the same func$^n$, it is known as recursion.
- The func$^n$ which calls the same func$^n$, is known as recursive func$^n$.
- A func$^n$ that calls itself and doesn't perform any task after func$^n$ call, is known as tail recursion. In this we generally call the same func$^n$ w/ return statement.

Ex— factorial, fibonacci sequence, Binary search, Tree & graph traversal

## * Storage class

It define the scope, visibility, lifetime and memory location of variables and func$^n$ within a prog.

C++ has 4 primary storage classes- (i) auto (ii) register (iii) static (iv) extern.

## * Func$^n$ overloading.

It is a feature that allows multiple ~~features~~ func$^n$ to have the same name but different parameters. This helps create multiple versions of a func$^n$ to handle different types or no. of inputs.

Examples—

(i) Diff no of parameters—

```
sum(int num1, int num2)
sum(int num1, int num2, int num3)
```

(ii) Diff parameter types—

```
sum(int num1, int num2)
sum(double num1, double num2)
```

(iii) Diff sequence of parameters-

```
sum(int num1, double num2)
sum(double num1, int num2)
```

* `int sum(int, int)` } This is not valid as parameter list is same.
`double sum(int, int)`

Date .....

Expt. No. ..... Page No. .....

* **friend function**

friend func$^n$ in c++ is a func$^n$ that is not a member of a class but is allowed to access the class's private and protected member.

By declaring a func$^n$ as a friend of a class, you enable it to access otherwise restricted data within the class.

* **Constructor.**

It is a special member func$^n$ of a class that initializes objects of that class. It has the same name as the class and is automatically called when object of the class is created.

→ constructors are essential for setting up initial values, allocating resources, or performing any startup task.

→ It do not have a return type, not even void.

Types -

(i) Default - Takes no argument and initializes objects with default values.

(ii) Parameterized - Takes parameters to allow different initial values for objects.

(iii) copy - Initializes an object by copying values from another object of the same class.

(iv) move - (c++ 11) → Transfer resources from a temporary object to a new object, improving efficiency

→ constructor play a vital role in resource management, ensuring objects are ready to use immediately after creation.

URBAN EDGE

Teacher's Signature : _____

## * Destructor

It is a special member func$^{n}$ of a class that is automatically called when an object goes out of scope or is explicitly deleted.

→ The purpose of destructor is to perform any necessary cleanup, such as releasing resources that the object may have acquired during runtime.

→ No parameter or return type

→ Only one destructor is allowed per class, it can't be overloaded.

→ prefixed with tilde ( ~ )

## * Dynamic constructor

- It is used to allocate the memory to the objects at runtime. Memory is allocated at runtime using 'new' operator.
- They are commonly used when the size of an array, buffer or other resource depends on runtime info.
- Require destructor.

## * constructor overloading.

More than one constructor with diff signature in a class

(i) No. of arguments
(ii) Types of argument
(iii) Sequence of arguments.

## * friend class.

A friend class can access private & protected members of other class in which it is declared as friend.

Date .....

Expt. No. ..... Page No. .....

## * Pointers.

- It is a variable that stores the memory address of another variable.
- It provide powerful capabilities for managing memory, enabling data manipulation and direct access to memory location.
- It is type-specific → They are specific to the data type they point to (eg. int*, float*), ensuring type safety.
- Dereferencing → Using the * operator, pointers can access the value stored at the memory address they hold.
- int* ptr declares a pointer to an Integer
  The & operator gets the address of a variable

Key use:-

(I) Dynamic Memory allocation (III) Passing by reference
(II) Array & String (IV) Func^n pointers.

## * Exceptional Handling

It is a mechanism that allows the program to handle unexpected situation, such as errors or unusual condition, during runtime without crashing.

key concepts-

(I) Try block - The try block contain code that might throw an exception
(II) Throw keyword - It is used to signal the occurance of an error "throwing" an exception.
(III) catch block - It catches and handles exception thrown by the try block.



Teacher's Signature : _____

## * Template

It allow writing generic and reusable code by enabling funcn and classes to operate with diff data types without being rewritten for each type.

Types-

(I) Funcn templates – Enable creating funcn that work with any data type.

(II) Class template – Allow creating classes that can handle multiple types.

Benefits–

(I) code reusability

(II) Type safety

(III) efficiency.

## * File Handling.

It allow programs to read from and write to files stored on disk.

It provides file handling through <fstream> library, which includes

(I) ifstream – input file stream

(II) ofstream – output file stream

(III) fstream – for both reading and writing.